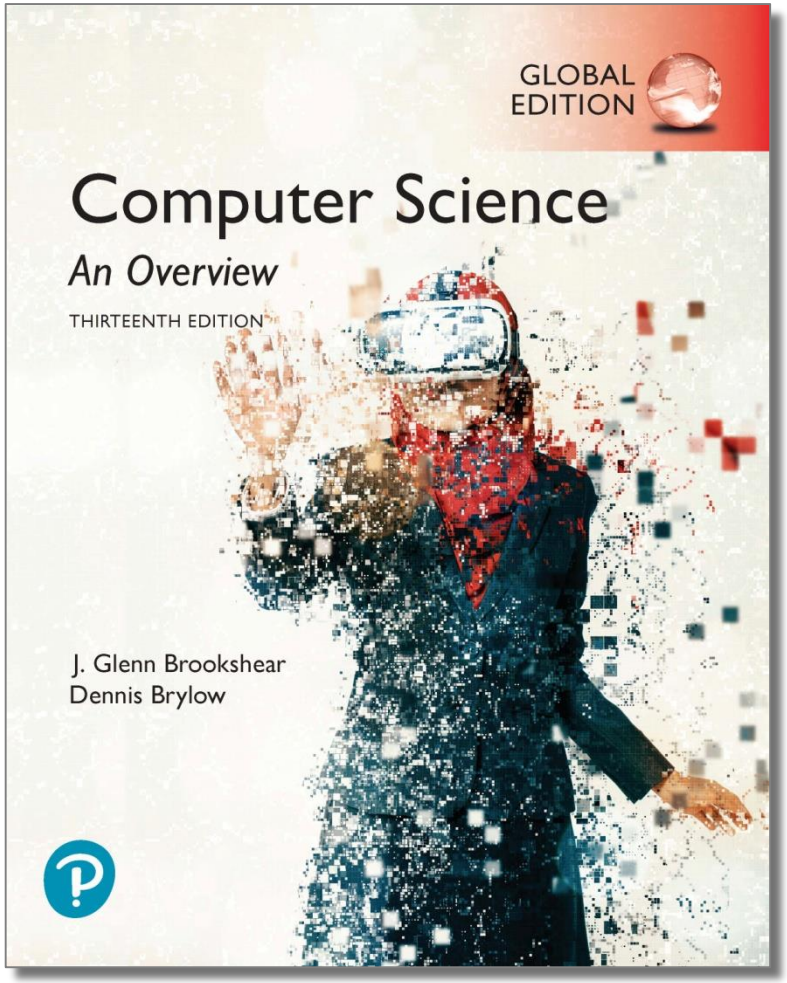
# Bilgisayar Bilimine Giriş

13. Baskı, Global Edition




## Bölüm 2

Veri İşleme

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Bölüm 2: Veri İşleme

- 2.1 Bilgisayar Mimarisi
- 2.2 Makine Dili
- 2.3 Program Yürütme
- 2.4 Aritmetik/Mantık
- 2.5 Diğer Cihazlarla İletişim
- 2.6 Veri İşlemeyi Programlamak
- 2.7 Diğer Mimariler

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

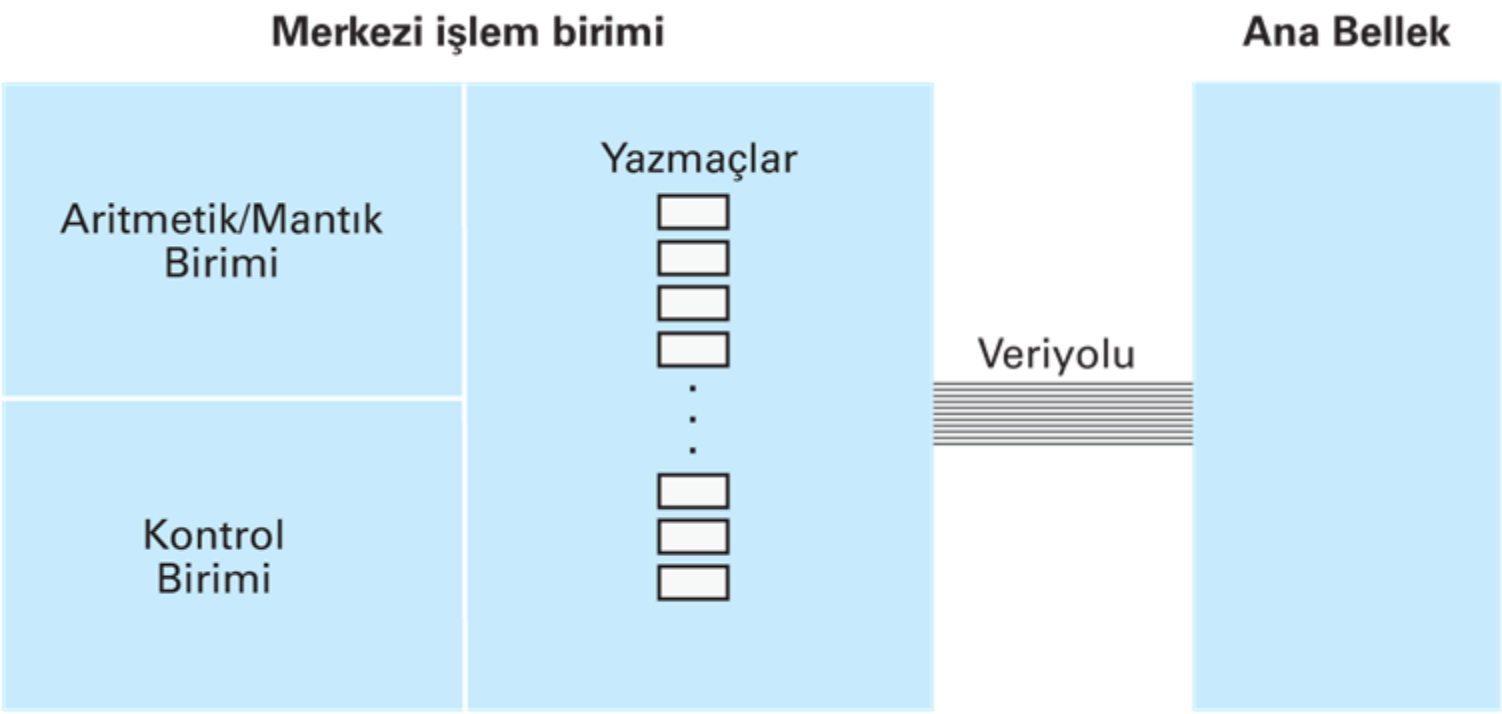
# 2.1 Bilgisayar Mimarisi

- Central Processing Unit (CPU) (Merkezi İşlem Birimi)
  - Aritmetik/Mantık Birimi
  - Control Birimi
  - Yazmaç Birimi (Register Unit)
    - Genel  yazmaçlar
    - Özel amaçlı yazmaçlar
- Veriyolu(Bus)
- Ana Bellek

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.1 Veriyolu(Bus) ile birbirine bağlanmış Ana Bellek ve Merkezi İşlem Birimi (CPU)




Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Depolanmış Program Kavramı

Bir program bit desenleri şeklinde kodlanıp Ana Bellekte depolanabilir. Kontrol Birimi tarafından çıkartılabilir, çözülebilir ve çalıştırılabilir.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# 2.2 Makine Dili

- **Makine talimatı:** Bir talimat (yönerge)CPU tarafından çalıştırılabilrn bir bit deseni olarak kodlanır.
- **Makine dili:** Makinenin anladığı bütün talimatların dizisidir.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Makine Dili Felsefeleri

- Azaltılmış Komut Kümeli Mikroişlemciler(Reduced Instruction Set Computing)(RISC)
- Az, basit, etkili ve hızlı komutlar
  - Örnekler: Apple/IBM/Motorola'nın PowerPC'si ve ARM
- Karmaşık Komut Kümeli Mikroişlemciler(Complex Instruction Set Computing) (CISC)
  - Fazla ,kullanışlı ve güçlü komutlar
  - Örnek: Intel

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Makine Komut Tipleri

- Veri Transfer: Veriyi bir yerden başka bir yere aktarma(Örnek: LOAD, STORE)
- Aritmetik/Mantık: Bit desenlerinde işlemler (Örnek: +, -, *, /, AND, OR, SHIFT, ROTATE)
- Kontrol: Programın direkt çalıştırılması (Örnek: JUMP, BRANCH)

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# 2.2 Bellekte depolanmış veride toplama

Adım 1. Eklenecek değeri bellekten al ve yazmaca yerleştir.

Adım 2. Eklenecek diğer değeri bellekten al ve diğer yazmaca yerleştir.

Adım 3. Adım 1 ve 2'de **girdi** olarak **kullanılan yazmaçlardaki bilgileri alarak toplama devresini aktifleştir ve sonucu** tutmak için tasarlanmış diğer yazmaca ver.

Adım 4. Sonucu bellekte depola.

Adım 5. Dur.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.3 Bellekte depolanmış değerde bölme

Adım 1. Bellekten bir sayıyı bir yazmaca YÜKLE

Adım 2. Bellekten başka bir sayıyı başka bir yazmaca YÜKLE

Adım 3. Eğer ikinci sayı 0 ise Adım 6'ya ATLA

Adım 4. İlk yazmaçtaki sayıyı ikinci yazmaçtaki sayıya böl ve sonucu bir üçüncü yazmaçta tut

Adım 5. Üçüncü yazmacın içeriğini belleğe KAYDET

Adım 6. Dur.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

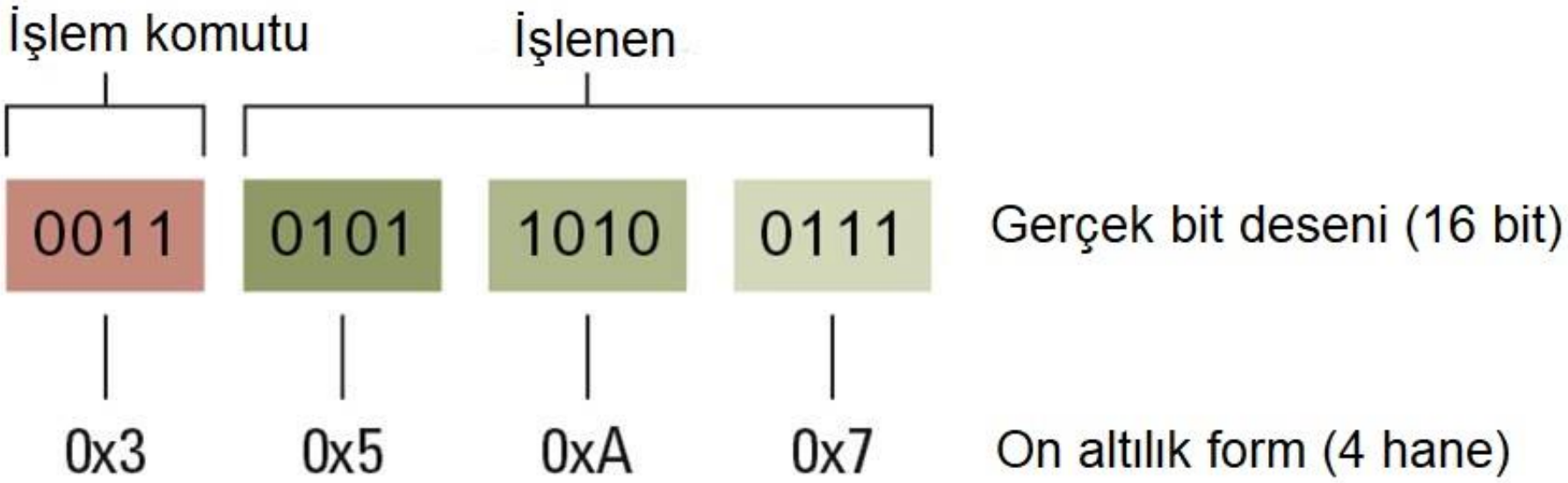
# Makine komutunun bölümleri

- **İşlem komutu(Op-code):** Uygulanacak işlemi belirtir
- **İşlenen(Operand):** İşlem hakkında daha kapsamlı bir bilgi verir
  - İşlenen değişkenler işlem komutuna göre yorumlanır

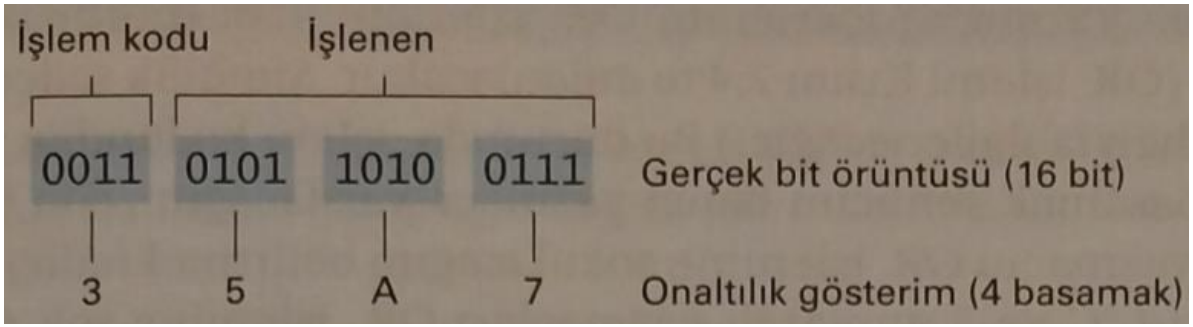
Pearson

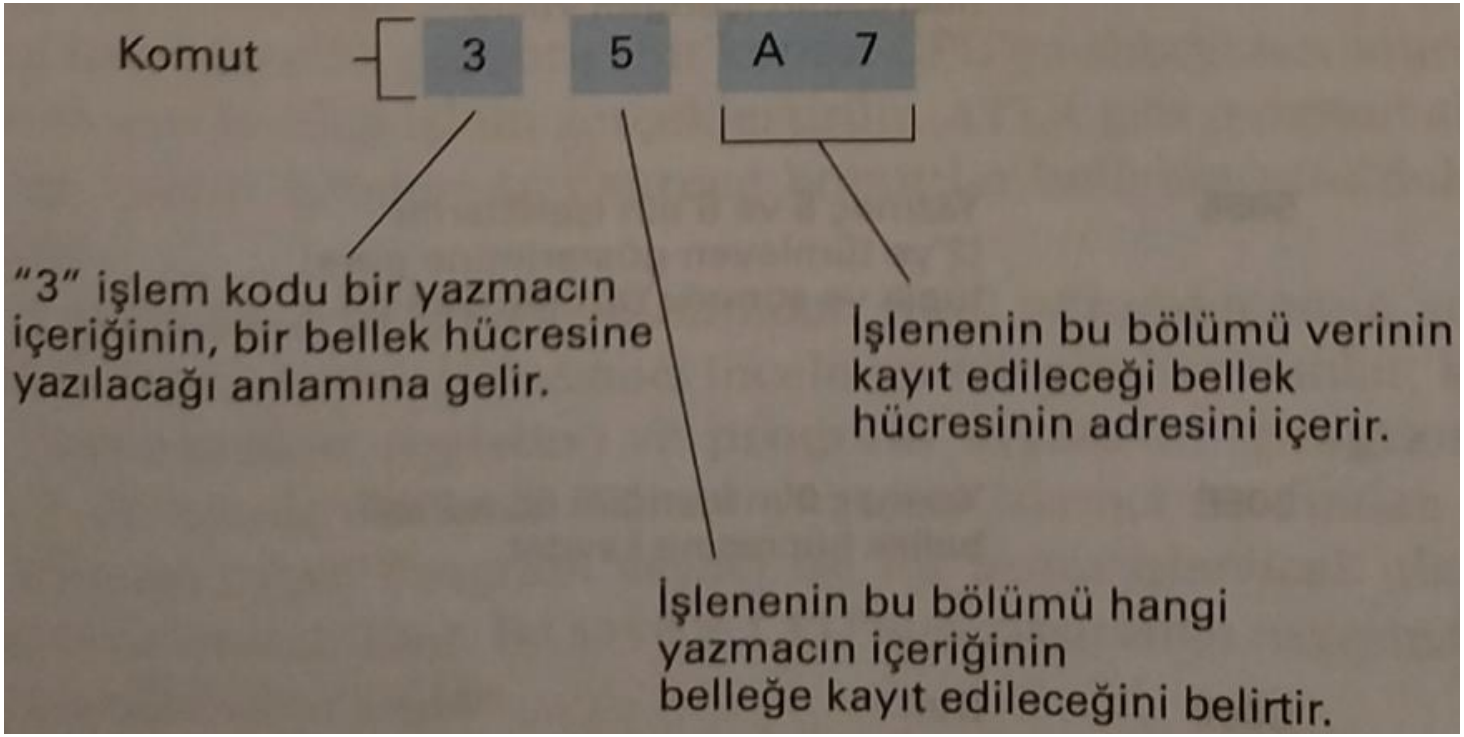
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.5 Bir Makine dili komutunun yapısı




Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.




Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

A simple machine language (from Computer science: An overview, by G. Brookshear and D. Brylow, 12th edition)

| Form of instruction | Meaning of the instruction |
|---|---|
| 1RXY | LOAD register R with the bit pattern found in the memory cell whose address is XY |
| 2RXY | LOAD register R with the bit pattern XY |
| 3RXY | STORE the bit pattern found in register R in the memory cell whose address is XY |
| 40RS | MOVE the bit pattern found in register R to register S |
| 5RST | ADD the bit patterns in registers S and T and leave the result in register R (the patterns in S and T are treated as 2's complement representations) |
| 6RST | ADD the bit patterns in registers S and T as though they represented values in floating-point notation and leave the floating-point result in register R |
| 7RST | OR the bit patterns in registers S and T and place the result in register R |
| 8RST | AND the bit patterns in registers S and T and place the result in register R |
| 9RST | XOR the bit patterns in registers S and T and place the result in register R |
| AR0X | ROTATE the bit pattern in register R one bit to the right X times. Each time place the bit that started at the low-order end at the high-order end |
| BRXY | JUMP to the instruction located in memory cell at address XY if the bit pattern in register R equals the bit pattern in register 0 |
| C000 | HALT the execution |

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# 2.3 Program Çalıştırma

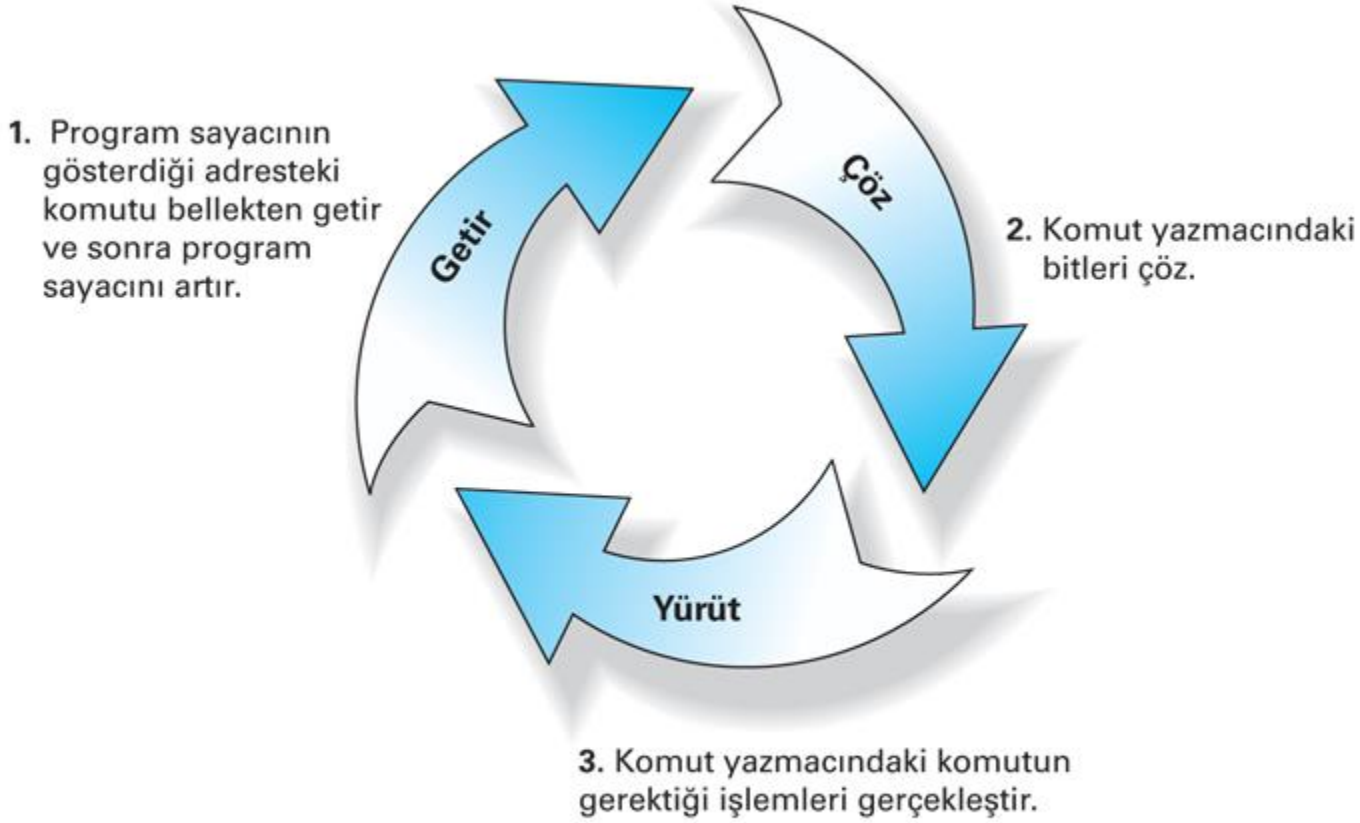
- İki özel amaçlı kayıt tarafından kontrol edilir
  - Komut yazmacı
    - Mevcut komutu tutar
  - Program sayacı
    - Sıradaki komutun adresini tutar

- Makine döngüsü: (bu üç adımı tekrarlar)
  - Bilgiyi getir,çözümle,yürüt(yap)

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

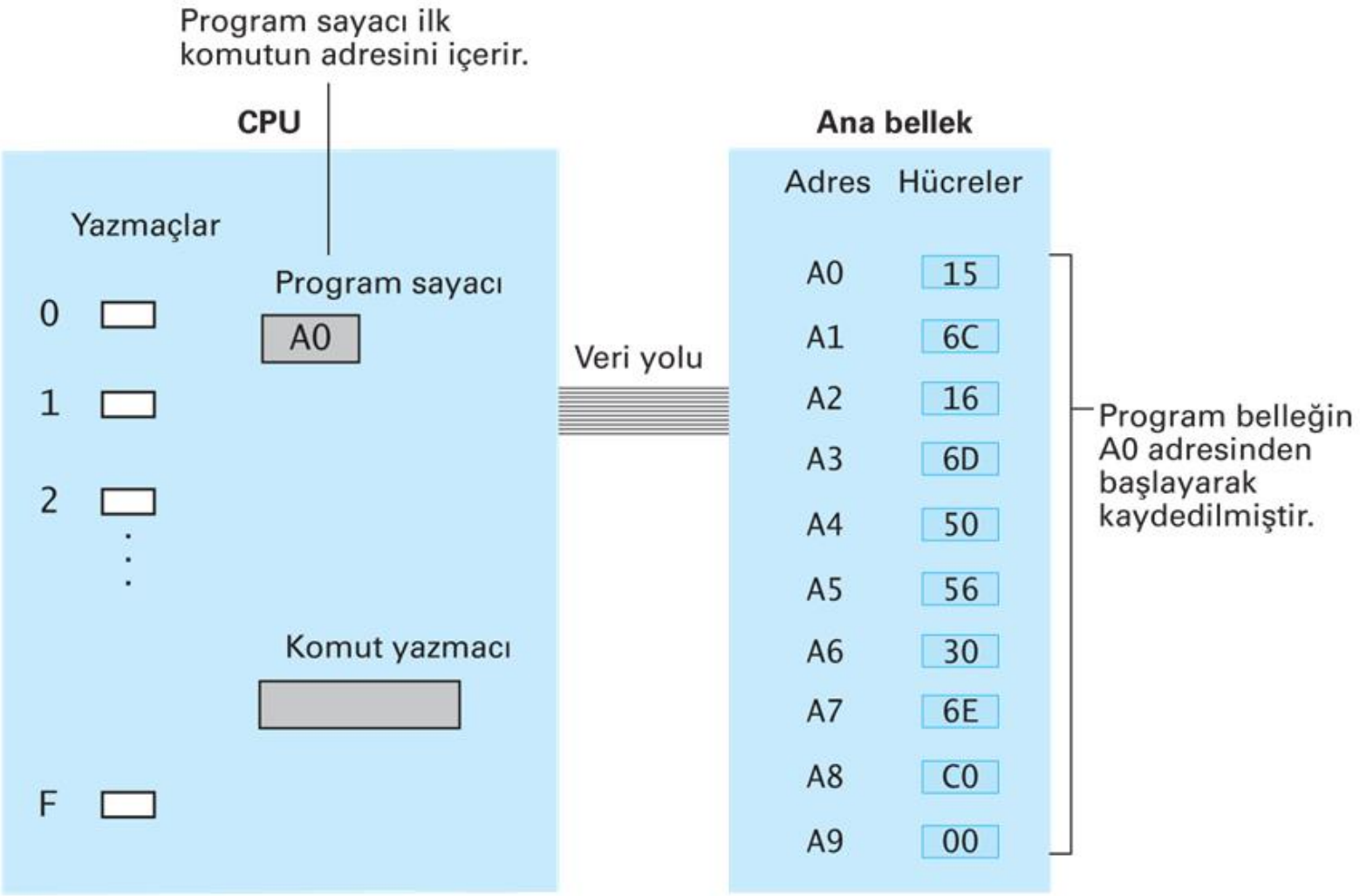
# Şekil 2.8 Makine döngüsü




Pearson

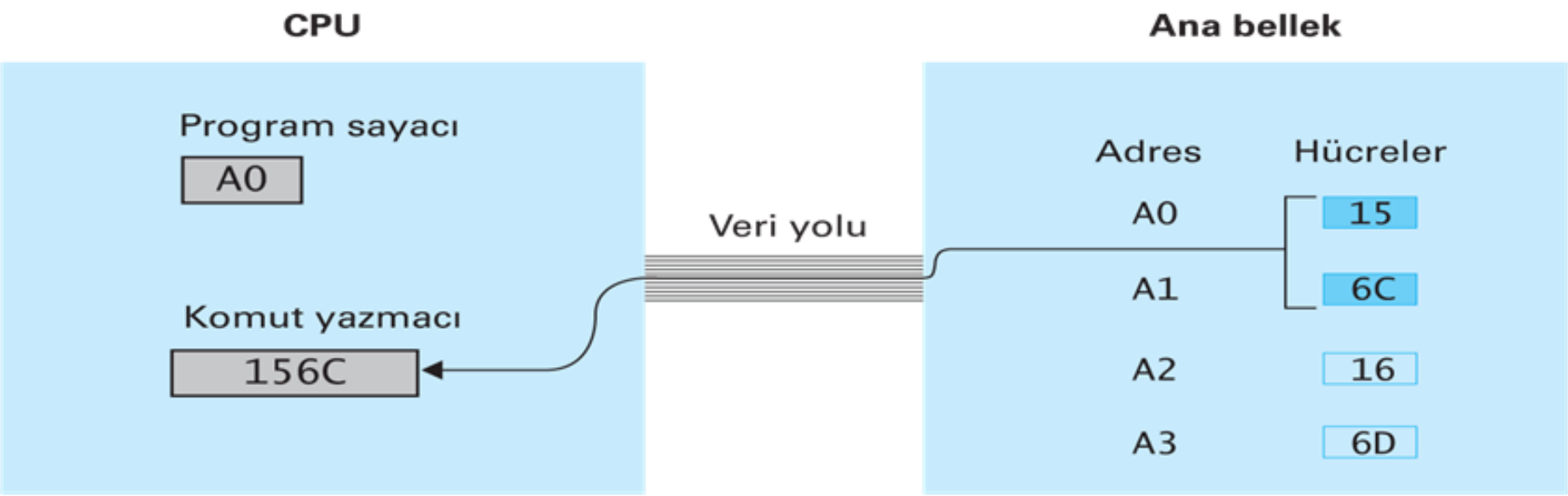
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.10 Şekil 2.7'deki program ana bellekte depolanmış ve yürütülmeye hazır




Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.11 Makine döngüsünün bilgiyi alma adımının işleyişi

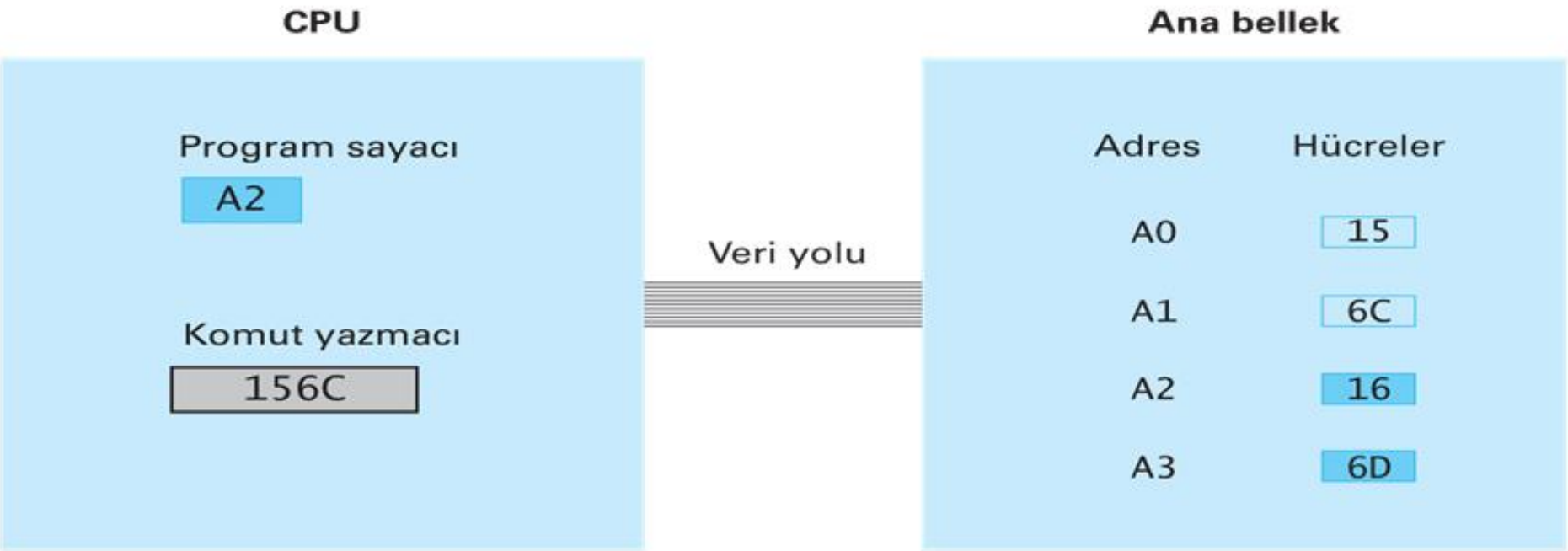



**a.** Getir adımının başında A0 adresinde başlayan komut bellekten okunur ve komut yazmacına yerleştirilir.

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.11 Makine döngüsünün bilgiyi alma adımının işleyişi(devamı)




**b.** Daha sonra program sayacı artırılarak bellekteki bir sonraki komutu

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# 2.4 Aritmetik/Mantık Komutları

- Mantık işlemleri:
  - VE(AND), VEYA(OR), Özel OR(XOR)
- Döndürme ve kaydırma işlemleri:
  - Dairesel kaydırma, mantıksal kaydırma, aritmetik kaydırma
- Aritmetik işlemler:
  - ekle, çıkar, çarp, böl

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# ARİTMETİK MANTIK Operasyonları

## Mantık İşlemleri

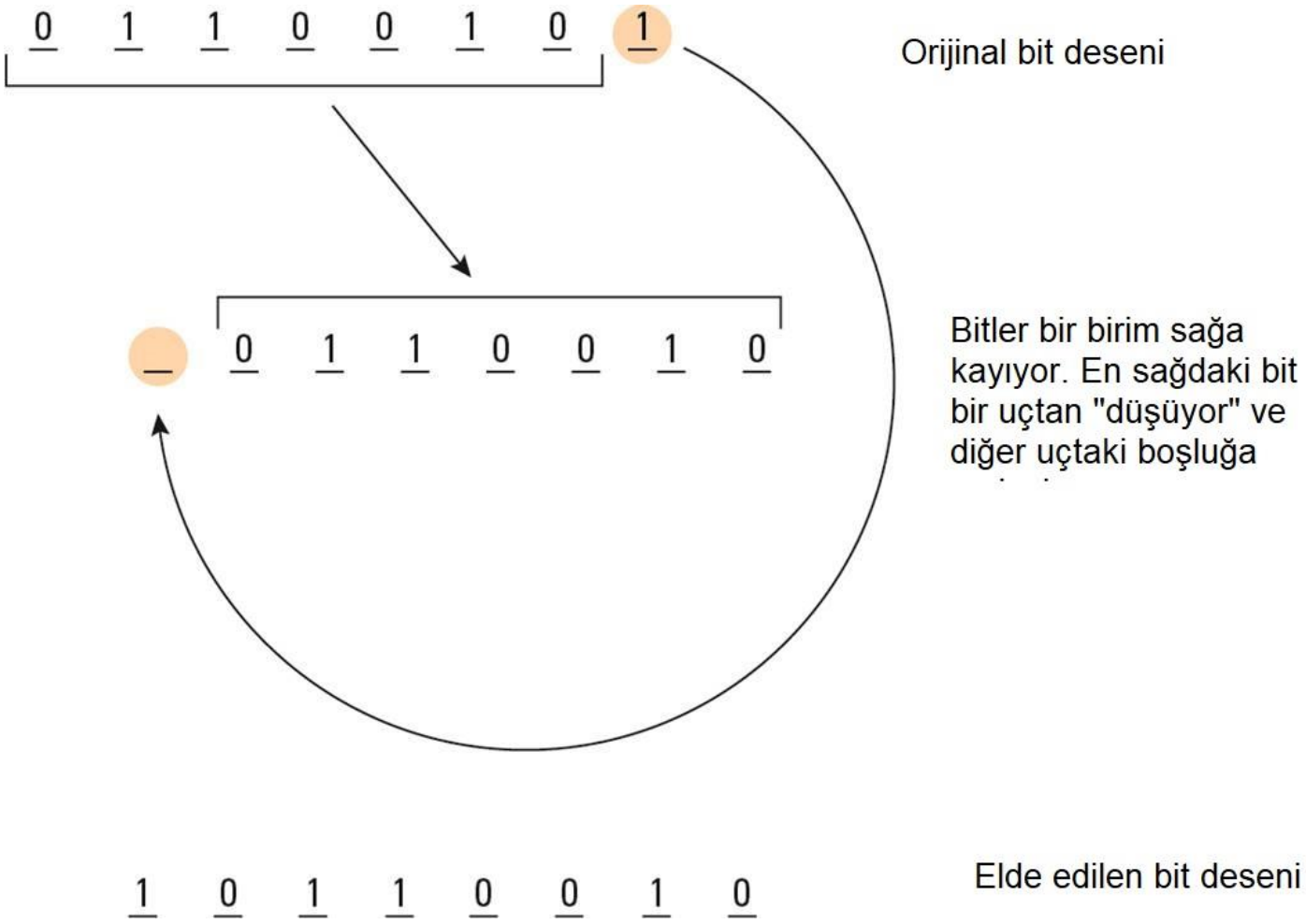
- Tek bir çıkış dizgesi oluşturmak için iki dizgenin bitlerini birleştiren **bitsel işlemler**e genişletilebilirler. İki dizgeyi bir mantık işlemine sokmak için dizgelerin aynı sıralarında bulunan bitler basit mantık işlemine sokulurlar.

```
     10011010        10011010        10011010
AND  11001001   OR   11001001   XOR  11001001
     10001000        11011011        01010011
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.12 0x65 bit desenini sağa doğru bir bit kaydırma işlemi




Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

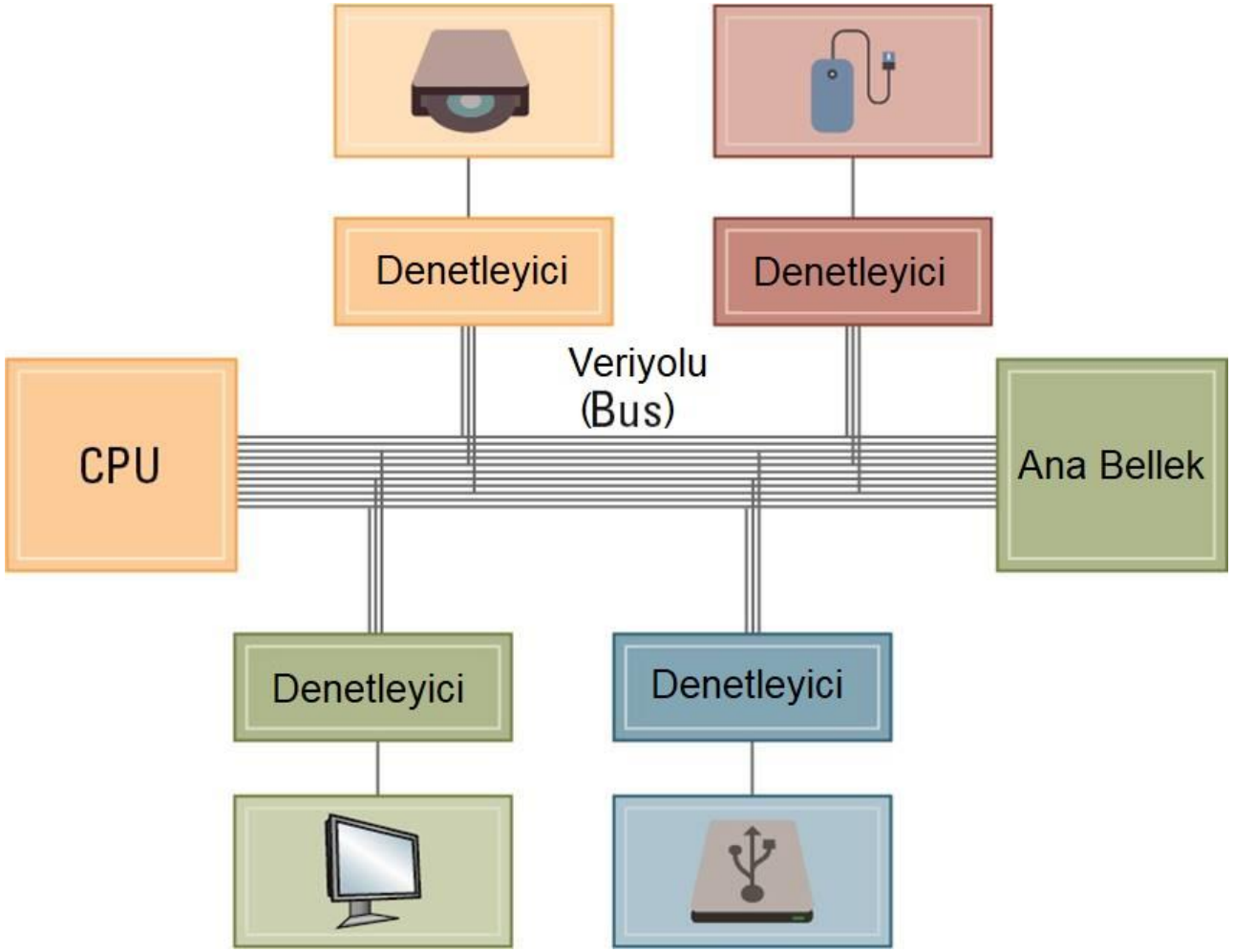
# 2.5 Diğer aygıtlarla iletişim

- **Denetleyici:** Bilgisayarın diğer aygıtlarla iletişimini yönetir
  - Özelleştirilmiş (aygıtın cinsine göre)
  - Genel amaçlı (USB, HDMI)
- **Kapı (Port):** Aygıtın bilgisayara bağlandığı yer
- **Bellek-haritalı giriş/çıkış:** CPU'ya bellek konumuymuş gibi gözüken aygıtlardır

Pearson

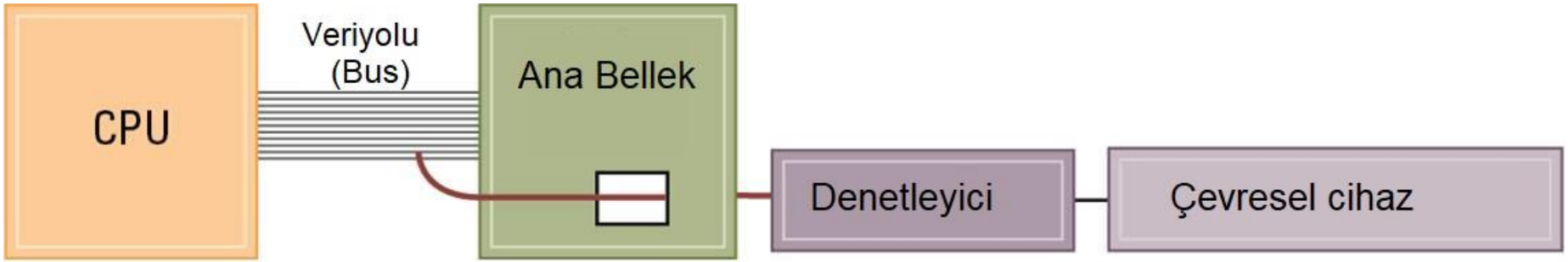
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.13 Makinenin veriyoluna bağlı Denetleyiciler




Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.14 Bellek-haritalı giriş/çıkış'ın görsel bir gösterimi




Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Diğer aygıtlarla iletişim (devamı)

- **Direkt bellek erişimi (DMA):** Ana belleğin veriyolu üstündeki bir denetleyici tarafından erişimi
- **El sıkışma:** Bilgisayar ve çevresel cihaz arasındaki veri transferi süreci

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Diğer aygıtlarla iletişim (devamı)

- **Popüler iletişim medyası**
  - **Paralel iletişim:** Çoklu sinyalin aynı anda, her biri ayrı “hatta”gitmesi (bilgisayarın iç veriyolu)
  - **Seri iletişim:** Sinyallarin teker teker aynı ve tek “hat” üzerinden gitmesi (USB, FireWire)

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Veri İletişim Oranları

- Ölçü birimleri
  - bps:  saniye başına gönderilen bit(bits per second)
  - Kbps:  Kilo-bps (1,000 bps)
  - Mbps:  Mega-bps (1,000,000 bps)
  - Gbps:  Giga-bps (1,000,000,000 bps)
- Bant-genişliği: Mümkün olan maksimum hız

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# 2.6 Veri programlama işlemi

- Programlama dilleri kullanıcıyı ana makine dilinin karmaşıklığından kurtarır:
  - Bir tek Python ifadesi onlarca belki de yüzlerce makine komut haritası kaplayabilir
  - Programcının, işlemcinin hangi mimariyi kullandığını bilmesine gerek yoktur(RISC yada CISC)

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Python Kodlama dilinde bit düzeni problemleri

```
print(bin(0b10011010 & 0b11001001))
#'0b10001000‘ Çıktısı verir
```

```
print(bin(0b10011010 | 0b11001001))
# '0b11011011'Çıktısı verir
```

```
print(bin(0b10011010 ^ 0b11001001))
# '0b1010011'Çıktısı verir
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Kontrol yapıları

- If ifadesi:

```
if (su_sicakligi > 140):
    print(‘Su cok sicak!')
```

- While ifadesi:

```
while (n < 10):
    print(n)
    n = n + 1
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Fonksiyonlar

- **Fonksiyon**: Verilen parametre(ler) üstünde gösterilmesi gereken işlem serisine verilen addır
- **Fonksiyon çıktısı**: İfadedeki fonksiyonun görünümü

```
x = 1034
y = 1056
z = 2078
biggest = max(x, y, z)
print(biggest)  # '2078‘ yazdırır
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Fonksiyonlar (devamı)

- **Argüman değeri**: Parametreye girilmiş değer
- Fonksiyonlar bir değere **geri döner**
- **void fonksiyonları**, veya **prosedürler,** bir değere geri dönmezler

```
sideA = 3.0
sideB = 4.0
# Pisago teoremini kullanarak üçüncü kenarın
uzunluğunu hesaplar
hypotenuse = math.sqrt(sideA**2 + sideB**2)
print(hypotenuse)
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Giriş / Çıkış

```
# Nizami bir üçgenin hipotenüsünü hesaplar

import math

# Kenar uzunluklarının girişi, ilk deneme
sideA = int(input(‘A kenarının uzunluğu
nedir?'))
sideB = int(input('B kenarının uzunluğu
nedir?'))

# Üçün kenarı Pisagor Teoremi ile hesaplar
hypotenuse = math.sqrt(sideA**2 + sideB**2)

print(hypotenuse)
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Maraton İdman Programı

```
# Maraton idman yardımcısı.

import math

# Bu fonksiyon dakikaları ve saniyeleri
# sadece saniye şekline dönüştürür.
def total_seconds(min, sec):
    return min * 60 + sec

# bu fonksiyon saatte alınan mil bilgisini kullanıp bir
milin kaç saniyede alınacağını hesaplar
def speed(time):
    return 3600 / time
```

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Maraton İdman Programı(devamı)

```
# Kullanıcıdan bir mil için gerekli süreyi ve mesafeyi
al.
pace_minutes = int(input('Minutes per mile? '))
pace_seconds = int(input('Seconds per mile? '))
miles = int(input('Total miles? '))

# Hızı hesapla ve yazdır.
mph = speed(total_seconds(pace_minutes, pace_seconds))
print(‘Hızınız ' + str(mph) + ' mph’dir')

# Çalışma için ayrılmış zamanı hesapla.
total = miles * total_seconds(pace_minutes, pace_seconds)
elapsed_minutes = total // 60
elapsed_seconds = total % 60

print(‘Çalışma süreniz ' + str(elapsed_minutes) +
              ' dakikadır ' + str(elapsed_seconds) + '
secs')
```

Pearson
Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# Şekil 2.15 Örnek maraton idman verileri

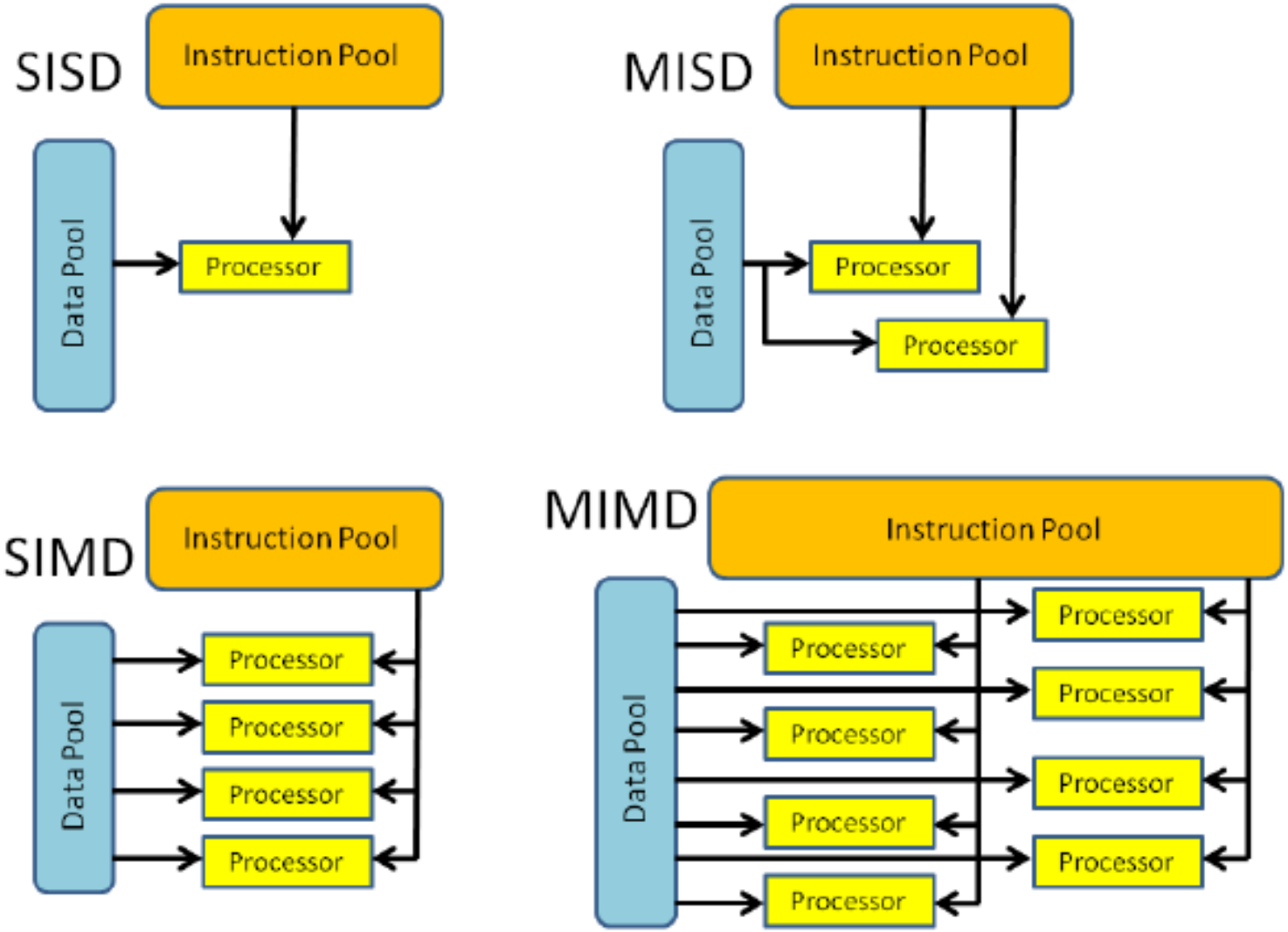
| Bir mil için geçen süre | | | | Toplam geçen süre | |
|---|---|---|---|---|---|
| Dakika | Saniye | Mil | Hız (mil/saat) | Dakika | Saniye |
| 9 | 14 | 5 | 6.49819494584 | 46 | 10 |
| 8 | 0 | 3 | 7.5 | 24 | 0 |
| 7 | 45 | 6 | 7.74193548387 | 46 | 30 |
| 7 | 25 | 1 | 8.08988764044 | 7 | 25 |

Pearson

Copyright © 2019 Pearson Education, Ltd. All Rights Reserved.

# 2.7 Diğer Mimariler

- Üretilen işi artıran teknolojiler:
  - Küme komut işleme (Pipelining): Makine döngülerinin birden fazlasının aynı anda çalıştırılması
  - Paralel işleme: Birbirine benzeyen birçok işlemci kullanır
    - SISD: Tek komut, Tek veri
      - Paralel işleme olmaz
    - MIMD: Çoklu komut, Çoklu veri
      - Farklı programlar, farklı veri
    - SIMD: Tek komut, Çoğul veri
      - Aynı program, farklı veri


SISD
Instruction Pool
Data Pool
Processor
MISD
Instruction Pool
Data Pool
Processor
Processor
SIMD
Instruction Pool
Data Pool
Processor
Processor
Processor
Processor
MIMD
Instruction Pool
Data Pool
Processor
Processor
Processor
Processor
Processor
Processor
Processor
Processor